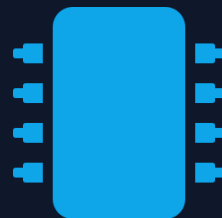


EDGE AI & MACHINE LEARNING INFERENCE

A Comprehensive Practical Guide



3–6 Hours

Duration

Windows + VS Code

Platform

YOLO + ONNX

Focus

Engineering

Level

Session Agenda

3–6 hours · Practical + Theory

01 Introduction to Edge AI



30 min

⌚ 30 min

05 Advanced Applications



60 min

⌚ 60 min

02 Theory & Architecture



40 min

⌚ 40 min

06 Tracking & Analytics



50 min

⌚ 50 min

03 Environment Setup



45 min

⌚ 45 min

07 Projects & Assessment



45 min

⌚ 45 min

04 YOLO Model Deployment



60 min

⌚ 60 min

08 Future of Edge AI



30 min

⌚ 30 min

Learning Objectives

By the end of this session, you will be able to:

LO1

Explain Edge AI principles and compare with cloud-based AI architectures

LO2

Set up a Python development environment with VS Code and virtual environments

LO3

Deploy pre-trained YOLO models for real-time object detection via webcam

LO4

Export models to ONNX format and benchmark inference performance

LO5

Build real-time applications: ROI detection, people counters, and tracking

LO6

Apply model optimization: quantization, pruning, and architecture selection

LO7

Create professional monitoring dashboards for Edge AI systems

LO8

Design and evaluate Edge AI projects for real-world deployment

Introduction to Edge AI

What Is Edge AI?



Edge AI deploys and executes AI algorithms directly on local hardware — microcontrollers, single-board computers, smartphones, and embedded systems — rather than relying on centralized cloud servers. This brings computation physically closer to the source of data.



Real-Time

Sub-50ms latency
for critical decisions



Privacy

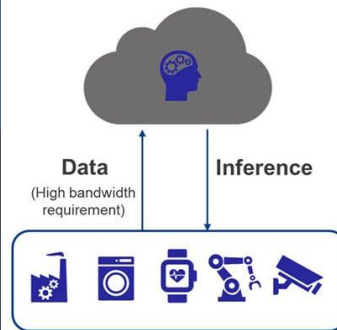
Data stays on device
never leaves premises



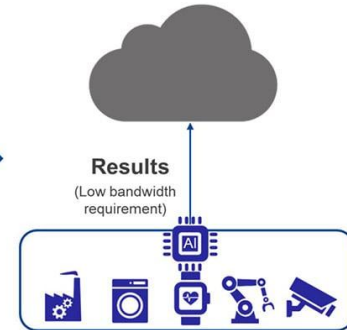
Offline

Works without
internet connectivity

Cloud AI – Inference in the cloud



Edge AI – Inference on Edge device



Key Applications:



Autonomous Vehicles



Industrial Robotics



Quality Inspection



Medical Monitoring



Smart Security

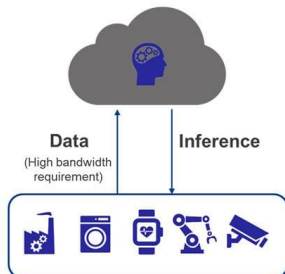


Agriculture

Edge AI vs. Cloud AI — Architectural Trade-Offs

Cloud AI

Cloud AI – Inference in the cloud



Latency 50–500+ ms (network round-trip)

Privacy Data leaves premises to remote servers

Bandwidth High — raw data must be uploaded

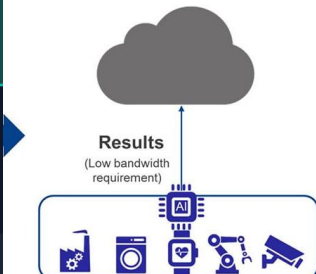
Cost at Scale Recurring API/GPU rental fees

Model Power Unlimited (cloud GPU clusters)

Scalability Elastic, on-demand scaling

Edge AI

Edge AI – Inference on Edge device



1–50 ms (on-device) 

Data stays on machine always 

Minimal — only metadata/alerts 

One-time hardware investment 

GPU/NPU-dependent 

Per-machine deployment 

✓ Edge wins for: Real-time control · Privacy-sensitive data · Remote/offline sites · Continuous high-throughput inference

Why Edge AI Matters — 4 Key Drivers

01 Latency Reduction



5–50 ms

On-Device

- Cloud AI: 50–500+ ms round-trip
- Critical for AV, robotics, surgery
- 100 ms extra latency = safety risk

03 Bandwidth & Offline



1.5 Gbps

1080p raw stream

- Edge sends only metadata/alerts
- Works in mines, ships, rural sites
- Air-gapped deployments supported

02 Privacy & Data Sovereignty



0 bytes

Sent to cloud

- HIPAA, GDPR, CCPA compliance
- Medical images stay on device
- No raw video transmitted

04 Cost Efficiency



\$0

Per inference

- No recurring API fees at scale
- One-time hardware investment
- Laptop = dev + deployment platform

Why Edge Inference Matters



Latency

<5ms

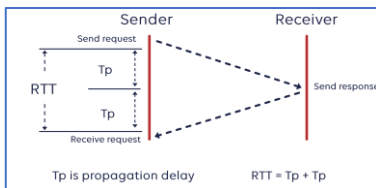
Edge inference
(on-device)

50-200ms

Cloud inference
(network round-trip)

Critical for real-time:

- Autonomous driving
- AR/VR pipelines
- Industrial robot control
- Medical monitoring



Eliminating network round-trips enables applications where milliseconds determine safety outcomes.

Why Edge Inference Matters



Privacy & Data Sovereignty



Data never leaves the device

Biometrics, health data, conversations stay local



Regulatory compliance

GDPR, HIPAA satisfied by design



Reduced attack surface

No cloud transmission = fewer interception vectors



User trust

Consumers demand on-device processing for personal data

Improve Security with Edge Computing and Edge AI



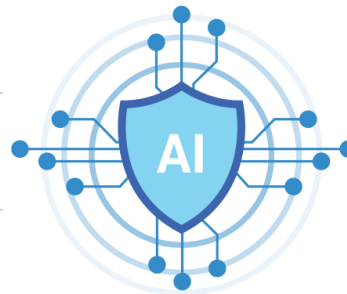
Edge data processing



Real-time threat detection



Instant response



Latency reduction



AI-based optimized video analysis



Increased data privacy



Why Edge Inference Matters



Bandwidth & Connectivity

75B+

IoT devices
by 2030

2.5 EB

Daily global
data generated

40%

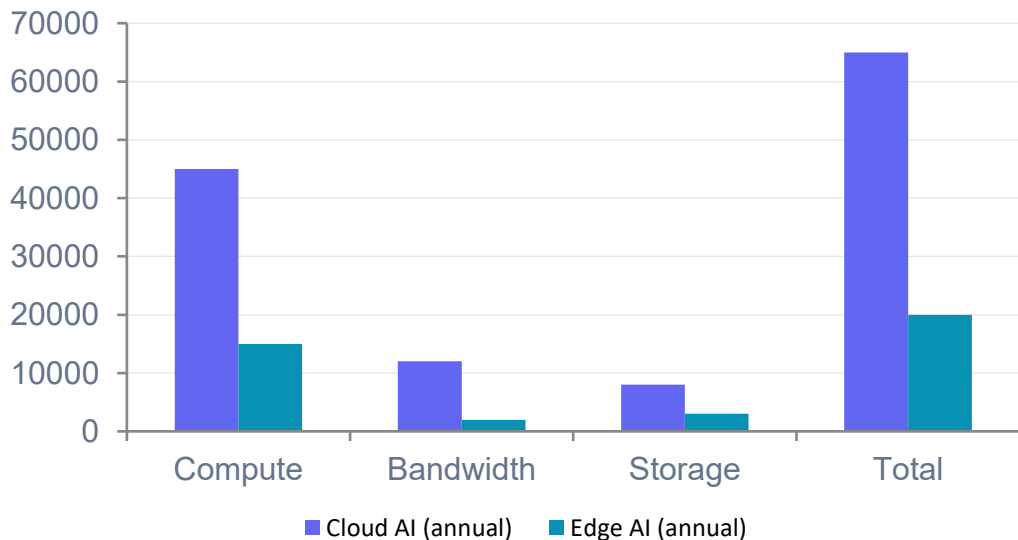
Unreliable
connectivity worldwide

Edge AI processes data locally, sending only results to the cloud. This dramatically reduces bandwidth requirements and enables disconnected operation.

Why Edge Inference Matters

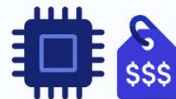


Cost Efficiency



Cost & Scalability

Edge = Higher
Upfront, Lower Ongoing



Low Ongoing Costs

Cloud = Lower
Upfront, Scales With Use



Costs Scale
With Usage

Key cost benefits:

- No per-inference API charges
- Reduced cloud provisioning
- Lower data transfer costs
- One-time HW vs. recurring cloud fees

Common Frameworks & Deployment Tools



Framework	Vendor	Target Platform
TensorFlow Lite	Google	Mobile, embedded, microcontrollers
ONNX Runtime	Microsoft	Cross-platform, multi-framework
TensorRT	NVIDIA	GPU-optimized inference
Core ML	Apple	iOS/macOS native inference
OpenVINO	Intel	Intel CPU/GPU/VPU optimization
MediaPipe	Google	Real-time ML pipelines

Choose based on target hardware, model format, and deployment constraints.

ML Lifecycle: Training vs. Inference

This guide focuses exclusively on the Inference phase — deploying pre-trained models to edge devices for real-time prediction.



TRAINING

Definition: Learning model parameters from labeled data

Compute: GPU clusters, TPU pods — extremely high

Data: Thousands to millions of labeled samples

Location: Cloud data centers, HPC clusters

Duration: Hours to weeks per training run

Goal: Maximize accuracy and generalization

Frequency: Periodic (retrain on new data batches)



INFERENCE

Definition: Using trained model to make predictions

Compute: Moderate to low — runs on edge devices

Data: Single sample or small batch at a time

Location: Edge devices, local machines

Duration: Milliseconds per prediction

Goal: Minimize latency, memory, and power

Frequency: Continuous (every sensor reading/frame)



Session Focus: Inference only — models are pre-trained; we deploy, optimize, and benchmark them on local hardware

The Edge AI Technology Stack



05 Integration Layer

MQTT, gRPC, REST, GPIO, USB — connects pipeline to sensors, actuators, dashboards



04 Application Layer

Orchestration: capture → preprocess → infer → post-process → visualize → alert

← Today



03 Model Layer

Pre-trained neural network: quantization, pruning, knowledge distillation applied

← Today



02 Runtime Layer

Inference engine: ONNX Runtime, TensorRT, TFLite, OpenVINO, CoreML, SNPE

← Today



01 Hardware Layer

Physical compute: CPU, GPU, NPU, TPU, FPGA, or dedicated AI accelerator chip

YOLO — You Only Look Once



The industry standard for real-time object detection at the edge — single forward pass through the network.



Single-Stage Detection

Predicts boxes AND class probabilities simultaneously in one pass — no proposal generation bottleneck



Anchor-Free (v8+)

Direct keypoint prediction — better generalization to unusual object shapes and aspect ratios



Multiple Sizes

Nano (3.2M params) to Extra-Large (68.2M params) — choose based on hardware constraints



Multi-Task Support

Detection, segmentation, pose estimation, classification, oriented bounding boxes — one API

Model Comparison

Model	Params	mAP50	CPU ms
-------	--------	-------	--------

YOLOv8n	3.2M	37.3	~80ms
---------	------	------	-------



YOLOv8s	11.2M	44.9	~130ms
---------	-------	------	--------

YOLOv8m	25.9M	50.2	~240ms
---------	-------	------	--------

YOLOv8l	43.7M	52.9	~400ms
---------	-------	------	--------

YOLOv8x	68.2M	53.9	~600ms
---------	-------	------	--------

★ We use YOLOv8n (Nano) for all lab exercises

SECTION 02 — Environment Setup



01 Install Python 3.11



- python.org/downloads → Python 3.11.x 64-bit
- CHECK ✓ 'Add python.exe to PATH'
- Verify: `python --version` → Python 3.11.x

⚠ Do NOT use Python 3.12+ (package compatibility issues)

02 Install VS Code



- code.visualstudio.com → latest stable
- Check 'Add to PATH' during install
- Install Python extension (ms-python.python)

⚠ Also install Pylance, Jupyter, GitLens extensions

03 Create Virtual Environment



- `mkdir edge-ai-project && cd edge-ai-project`
- `python -m venv venv`
- `.\venv\Scripts\Activate.ps1` (PowerShell)

⚠ If error: `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`

04 Install Dependencies



- `pip install ultralytics opencv-python numpy`
- `pip install torch onnxruntime matplotlib psutil`
- `pip install -r requirements.txt`

⚠ GPU users: `pip install onnxruntime-gpu` (then test with `nvidia-smi`)

Environment Verification — 4 Tests

Step 1 `gpu_test.py`

PyTorch & GPU check

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
```

✓ Expected: PyTorch version: 2.x.x
CUDA available: True (or False)

Step 2 `nvidia-smi`

NVIDIA driver check

```
nvidia-smi
```

✓ Expected: GPU table with name, driver, CUDA version
→ 'not recognized' = no GPU (use CPU mode)

Step 3 `test_opencv.py`

OpenCV & webcam check

```
import cv2
print(cv2.__version__)
cap = cv2.VideoCapture(0)
if cap.isOpened(): print('Webcam OK')
```

✓ Expected: OpenCV version: 4.x.x
Webcam OK — Frame size: 640x480

Step 4 `test_yolo.py`

YOLO model check

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
print(f'{len(model.names)} classes')
```

✓ Expected: Model loaded: 80 classes
Inference OK — 0 detections on noise

SECTION 03 — YOLO Model Deployment



LAB 6.1 Model Exploration

File: lab_6_1_model_info.py

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')

params = sum(p.numel() for p in
             model.model.parameters())

print(f'Classes: {len(model.names)}')
print(f'Params: {params:,}')
```

✓ 80 classes · 3,157,200 total parameters · YOLOv8n on COCO 2017

LAB 6.2 Real-Time Webcam Detection

File: lab_6_2_webcam_detection.py

```
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
```

```
while True:
    ret, frame = cap.read()
    results = model(frame, conf=0.5)
    annotated = results[0].plot()
    cv2.imshow('Detection', annotated)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

✓ Live annotated window · FPS counter · Press Q to quit

Neural Network Inference Pipeline



Troubleshooting Tips:

Black screen → try camera index 1 or 2. | Slow FPS → reduce frame resolution or confidence threshold. | Camera busy → close Zoom/Teams/OBS before running.

ONNX Export & Performance Benchmarking



What is ONNX?



Runtime optimization: operator fusion, constant folding, memory planning



Cross-platform: Windows, Linux, macOS, Android, iOS — one file



Hardware acceleration: CUDA, TensorRT, DirectML, OpenVINO, CoreML

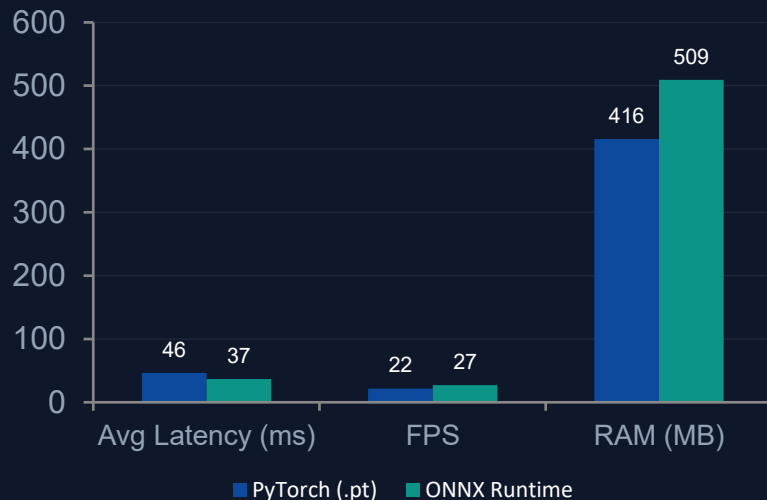


Small footprint: ~50 MB runtime vs ~2 GB full PyTorch

Lab 6.3 — Export to ONNX

```
model = YOLO('yolov8n.pt')
export_path = model.export(
    format='onnx',
    imgsz=640,
    simplify=True,
    opset=17,
    dynamic=False
)
```

Lab 6.5 — Benchmark Results (CPU Example)



ONNX: 20% faster latency · 26% higher FPS · works without PyTorch on deployment device

Discussion Questions

Q1: .pt vs .onnx — what's stored differently? **Q4:** Deploy to Raspberry Pi: PyTorch or ONNX? **Q5:** Why discard first 10 warm-up runs?

Model Optimization for Edge Deployment



Quantization

Reduce numerical precision

FP32 → FP16

50% size reduction, <1% accuracy loss. Default GPU start

FP16 → INT8

75% reduction, 2–4× speedup. Needs 100–1000 calibration images

Dynamic Quant

No calibration needed. Smaller speedup than static INT8



Pruning

Remove redundant weights

Structured

Remove whole filters/layers. Reduces FLOPs directly

Unstructured

Zero individual weights. Needs sparse hardware for speedup

Global Mag.

Remove lowest-magnitude weights globally across all layers



Knowledge Distillation

Teacher → Student transfer

Soft Labels

Student learns from teacher's probability distribution (dark knowledge)

Feature Mimic

Match intermediate activations, not just final output logits

Result

Compact student outperforms same-arch model trained from scratch



Architecture Selection

Pick edge-optimized models

MobileNetV3

Depthwise separable convolutions. Ideal for mobile CPU inference

EfficientNet-Lite

NAS-optimized scaling. Best accuracy-per-FLOP on constrained devices

YOLO Nano

3.2M params, 6.2 MB — runs at 30+ FPS on a laptop CPU

SECTION 04 — Advanced Real-Time Applications



LAB 7.1 Interactive ROI Detection

- Draw ROI rectangle with mouse click-drag
- Only count/detect objects inside ROI
- Live dashboard: FPS, confidence, object count
- Display ROI size and area in real-time
- Class statistics panel (person: 3, chair: 1...)
- Save snapshots with S key
- CSV logging of all detections with timestamp
- Adjustable confidence: UP/DOWN arrow keys

Keyboard Controls

Mouse drag: Draw ROI

R: Reset ROI

S: Save snapshot

↑/↓: Confidence ±0.05

Q: Quit

LAB 7.3 People Counter with Alert

```
ALERT_THRESHOLD = 3 # Max people
TARGET_CLASS = 0    # person

model = YOLO('yolov8n.pt')
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)

while True:
    results = model(frame, conf=0.5)
    count = sum(1 for b in results[0].boxes
                if int(b.cls[0]) == TARGET_CLASS)
    alert = count > ALERT_THRESHOLD
    status = 'ALERT: OVER CAPACITY'
    if alert:
        cv2.rectangle(frame, (0,0), ..., RED, 8)
```

Normal: count ≤ threshold · green border · status OK

OVER CAPACITY: count > threshold · red flash border · alert text



Production Use Cases

Retail occupancy · warehouse safety zones · clean room limits · MQTT alert triggers
· PLC relay activation

SECTION 05 — Object Tracking & Video Analytics



Detection vs. Tracking — Key Differences

Capability	Detection Only	Detection + Tracking
Per-frame objects	✓ Bounding boxes + class labels	✓ Plus persistent unique ID per object
Identity across time	✗ Same object = new box each frame	✓ Object keeps ID until it leaves scene
Unique counting	✗ Unreliable — counts same object repeatedly	✓ Accurate — each object counted once
Path / trajectory	✗ Not possible	✓ Records position history per ID
Dwell time	✗ Not possible	✓ First-seen to last-seen timestamp

LAB 8.1 Real-Time Multi-Object Tracking

```
results = model.track(frame,
    persist=True,
    conf=0.5)

# Draw trails
for box, tid in zip(boxes, ids):
    track_history[tid].append(cx, cy)
    cv2.line(frame, pts[i-1], pts[i],
        (0, alpha, 255), 2)
```

LAB 8.2 Bidirectional Line-Crossing Counter

```
LINE_Y = frame.shape[0] // 2
OFFSET = 30

if prev_y < LINE_Y - OFFSET
    and curr_y >= LINE_Y - OFFSET:
        count_down += 1
        counted_ids.add(tid)

cv2.line(frame, (0, LINE_Y),
    (w, LINE_Y), YELLOW, 3)
```

SECTION 06 — Professional Dashboard Development



Why Dashboards Matter

Operators, engineers, and managers need real-time performance metrics, detection statistics, and system health in an intuitive visual format — transforming raw telemetry into actionable insight.



Live Video Panel

Annotated camera feed with real-time bounding boxes and class labels



Performance Metrics

FPS, latency histogram, CPU gauge, RAM utilization bars



Detection Statistics

Per-class counts, confidence distribution, objects-over-time chart



System Health Panel

GPU temp, memory pressure, disk usage, uptime, alert indicators

Lab 9.1 — Progress Bar Helper

```
def draw_bar(canvas, x, y, w, h,
              value, max_val, color, label):
    fill_w = int(w * min(value/max_val, 1.0))
    cv2.rectangle(canvas, (x,y), (x+w,y+h),
                  DARK_BG, -1)          # background
    cv2.rectangle(canvas, (x,y), (x+fill_w,y+h),
                  color, -1)           # fill
    cv2.putText(canvas, f'{label}: {value:.1f}',
                (x, y-5), FONT, 0.45, LIGHT, 1)
```

Dashboard Design Best Practices



Update Rate: smooth FPS with 5–10 frame moving average



Color Coding: green=normal, yellow=warning, red=critical



Information Hierarchy: critical info at top (FPS, alerts)



Dark Theme: standard for control rooms, reduces eye strain



Responsive Layout: relative positioning for any resolution

SECTION 07 — Capstone Deployment Projects



Complete at least 2 of 3 projects. Each integrates multiple techniques into a portfolio-ready application.

01

Intermediate



Smart Parking Occupancy Monitor

Monitor a parking area, count vehicles, display occupancy with color-coded status, log to CSV

Requirements:

- Detect car/truck/motorcycle/bus in defined ROI
- Color-coded status: green/yellow/red by % full
- Log occupancy every 5 seconds to CSV
- Side panel: count, capacity, utilization %

Skills used:

ROI filtering (Lab 7.1) · Class counting (Lab 7.3) · CSV logging · OpenCV panels

02

Intermediate



Industrial PPE Safety Inspector

Detect workers, verify hard hat compliance, alert on violations, generate audit-ready CSV report

Requirements:

- Detect 'person' class (class 0)
- Check upper-body region for helmet overlap
- Green/red border for compliant/non-compliant
- Log violations to violations/ directory

Skills used:

Detection pipeline · Bounding box geometry · Alert system · Session reports

03

Advanced



Retail Customer Flow Analytics

Track shoppers, count entries/exits via line-crossing, measure dwell time, generate analytics report

Requirements:

- YOLO tracking with persistent IDs
- Bidirectional line-crossing counter
- Dwell time per unique ID (first/last seen)
- Post-session CSV + matplotlib histogram PNG

Skills used:

model.track() (Lab 8.1) · Line crossing (Lab 8.2) · Dwell analysis · Matplotlib charts

SECTION 08 — Emerging & Future Edge AI



Neural Processing Units (NPUs)

Now → 2027

Intel Meteor Lake, AMD Ryzen AI, Apple Neural Engine, Qualcomm Hexagon — 10–45 TOPS at 5–15 watts. Every laptop by 2027 will have built-in AI acceleration for YOLO-class models at 30+ FPS.



TinyML

Production now

Neural networks on microcontrollers with 256 KB RAM. TensorFlow Lite Micro, Edge Impulse, Apache TVM. Hearing aids, smart sensors, predictive maintenance, agriculture monitoring — under \$5 hardware.



Federated Learning at the Edge

Pilot → 2026

Train across many devices without centralizing data. Each device shares only weight gradients. Preserves privacy, reduces bandwidth. Google Gboard and Apple Siri use this today in production.



On-Device LLMs

Expanding 2025–2027

Llama 3 (8B) running on 8 GB RAM laptops via 4-bit quantization (GPTQ, AWQ). Private AI assistants, offline document analysis, code completion — no cloud needed. Maturing fast.

Quick Reference — Commands & Resources

Environment Commands

```
python -m venv venv

.\venv\Scripts\Activate.ps1

pip install -r requirements.txt

pip install torch torchvision --index-url
...

pip install onnxruntime-gpu
```

Verification Scripts

```
python gpu_test.py

nvidia-smi

python test_opencv.py

python test_yolo.py

python lab_6_1_model_info.py
```

YOLO Key APIs

```
model = YOLO('yolov8n.pt')

results = model(frame, conf=0.5)

results = model.track(frame,
persist=True)

model.export(format='onnx', imgsiz=640)

session =
ort.InferenceSession('model.onnx')
```

Reference Documentation



Ultralytics YOLOv8 Docs

docs.ultralytics.com



OpenCV Python

docs.opencv.org



ONNX Runtime

onnxruntime.ai/docs



PyTorch

pytorch.org/docs



NVIDIA TensorRT

developer.nvidia.com/tensorrt



Edge Impulse (TinyML)

docs.edgeimpulse.com

Session Complete

You can now independently design, build, deploy, benchmark, and optimize a production-grade Edge AI inference pipeline on a standard Windows workstation — with zero cloud dependencies.

— Session Competency Target

Key Takeaways



Edge AI runs inference on local hardware
— no cloud required



YOLOv8n delivers real-time detection at
20–30 FPS on CPU alone



ONNX Runtime: lighter, faster, cross-
platform inference engine



FP16 quantization: 50% size reduction,
<1% accuracy loss



`model.track()` adds persistent IDs for
counting and analytics



NPUs in every laptop by 2027 — Edge AI
is the career frontier